

CodeLab4 - Feature Extraction

Feature engineering is the art and science of selecting, transforming, and creating meaningful input features from raw data. Machine learning algorithms can extract valuable patterns and make accurate predictions. This exercise focuses on biomedical images derived from three distinct modalities: chest X-rays, abdomen and head CT scans. [1]–[3].

Feature extraction tasks heavily rely on the type of data and the objective. For instance, object detection tasks rely on edge detection, blob detection, and shape descriptors. For more complex tasks, e.g., ML aided diagnosis of pneumonia, convolutional neural networks (CNN) are often employed due to their high performance in image processing. The multi-layer structure remains identical as in neural network, with the main difference being in the learned parameters, which are the kernel weights, as opposed to the neuron weights. Kernels are small windows (filters), which overlap to the input image/data. The element wise products between the kernel weights and the overlapping portion of the input image are added together to produce a new feature map. After each summation step, the filter slides to the next region of the image.

In this lab, you will use a set of selected biomedical images to classify and categorize the dataset. Meanwhile, the effect of the feature selection on the image dataset will be discussed. You will also tell the differences before and after feature engineering. Hyperparameter tuning is also an important factor in the training process of modern ML workflow. A grid search method will be deployed to find the optimal model parameters for this specific task.

Provide your answers in the Jupyter Notebook either as a comment or insert a markdown cell. Please print out your Jupyter Notebook in PDF format (or HTML) and upload it to Brightspace. You will get points for each part separately, and you will get the grades based on your overall performance.

```
In [1]: import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import random
import numpy as np
import cv2

np.random.seed(42)
random.seed(42)
```

Task A - Dataset preparation (1 point)

This cell checks if a directory named "dataset" exists. If it does not exist, it opens the zip file named "dataset.zip" and extracts its contents into the "dataset" directory.

```
In [2]: import os
import zipfile

if not os.path.exists("dataset"):
    with zipfile.ZipFile("dataset.zip", "r") as zip_ref:
        zip_ref.extractall("dataset")
else:
    print("Dataset already exists, skipping unzip")
```

Dataset already exists, skipping unzip

Create the path to the dataset folder by merging the current working directory and the folder name. The variable `data_dir` that stores the path to a directory called `dataset\` within the current working directory.

NOTE: to run this cell in Vocareum/Google Colab/MacOS/Linux the directory name is `dataset/`, if you run it locally on Windows systems use `dataset\`

```
In [3]: data_dir = os.path.join(os.getcwd(), 'dataset/')
print('Number of images in the dataset folder: ', len(os.listdir(data_dir)))
```

Number of images in the dataset folder: 400

The folder should contain 400 items, if this is not the case, go back to jupyter interface, delete the dataset folder, and retry from the start of the notebook.

- A1) Complete the `load_data` function. It loads the data from the dataset folder. Four different classes are present in the dataset, and labels will be assigned according to their filenames. Make sure all three different formats of images are imported. The function `fnmatch` is used to search for a specific string in the filenames, labels should be assigned as follows:

Abdomen CT scans -> Class 0
Brain CT scans -> Class 1
Chest X-Rays normal -> Class 2
Chest X-rays with pneumonia -> Class 3

```
In [4]: from fnmatch import fnmatch

def load_data(directory):
    images = []
    labels = []
    for filename in os.listdir(directory): ## iterate over each file in directory
        if filename.endswith(".jpg") or filename.endswith(".png") or filename.endswith(".jpeg"): ## check if it is an image
            img = mpimg.imread(directory + filename) ## read image
            images.append(img)

            ## assign labels based on filename
            if fnmatch(filename, "Abdomen*"):
                labels.append(0)
            elif fnmatch(filename, "Brain*"):
                labels.append(1)
            elif fnmatch(filename, "Chest normal*"):
                labels.append(2)
            elif fnmatch(filename, "Chest Pneumonia*"):
                labels.append(3)
            else:
                raise ValueError('Unknown class: %s' % (filename))

    return images, np.asarray(labels)
```

- A2) Load the data using the load_data() function and verify the sizes of the dataset. It should create 400 samples and 400 labels.

```
In [5]: raw_images, labels = load_data(data_dir)

print(len(raw_images))
print(len(labels))

400
400
```

- A3) Plot a random image for each class in the same figure

Hint: you can use the function random.choice() to pick from the indexes that correspond to a specific class (np.where(condition))

```
In [6]: plt.figure(figsize=(7,7))
plt.suptitle('Sample images from each class',fontsize=15)

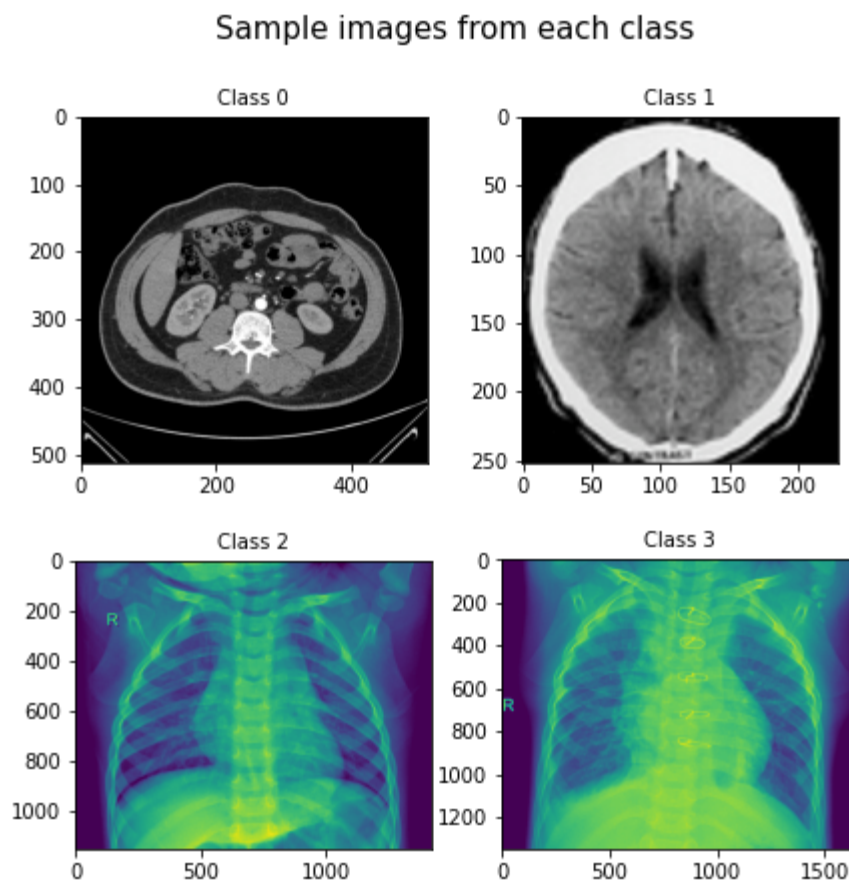
random_index_class0 = random.choice(np.where(labels == 0)[0])
plt.subplot(2, 2, 1)
plt.imshow(raw_images[random_index_class0])
plt.title('Class 0', fontsize=10)

random_index_class1 = random.choice(np.where(labels == 1)[0])
plt.subplot(2, 2, 2)
plt.imshow(raw_images[random_index_class1])
plt.title('Class 1', fontsize=10)

random_index_class2 = random.choice(np.where(labels == 2)[0])
plt.subplot(2, 2, 3)
plt.imshow(raw_images[random_index_class2])
plt.title('Class 2', fontsize=10)

random_index_class3 = random.choice(np.where(labels == 3)[0])
plt.subplot(2, 2, 4)
plt.imshow(raw_images[random_index_class3])
plt.title('Class 3', fontsize=10)
```

Out[6]: Text(0.5, 1.0, 'Class 3')



As you can see, the images have different characteristics: some are RGB, some others are B&W, and all have varying shapes. For this reason you're going to resize them and convert them in grayscale.

The `convert_to_grayscale` function takes an image as input and converts it to grayscale using the `cv2.cvtColor` function from the OpenCV library. It uses the `cv2.COLOR_BGR2GRAY` flag to specify the conversion from BGR color space to grayscale. The function then returns the grayscale image.

Only the RGB images need to be converted to grayscale, otherwise `cv2` will raise an error, so check for this verifying the shape of each image: RGB images can have 3 or 4 channels, one for each colour + transparency in the case of .png files.

The `resize_images_in_list` function takes a list of images and resizes each image to a specified target size. If the image has 3 or 4 channels (indicating it is a color image), it first converts the image to grayscale using the `convert_to_grayscale` function. Then, it resizes the image using the `cv2.resize` function and appends the resized image to a list. Finally, it returns the resized images as a numpy array. The default target size is set to (128,128) if no target size is specified.

```
In [7]: import cv2

def convert_to_grayscale(image):
    return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) ## convert the image to grayscale

def resize_images_in_list(image_list, target_size=(128,128)): ## default target size is 128,128

    resized_images=[]

    for image in image_list:    ## iterate through the list of images

        if image.shape[-1] == 3 or image.shape[-1] == 4: ## check if the last dimension of each image is 3 or 4
            image=convert_to_grayscale(image) ## if needed, convert the image to grayscale

            resized_image = cv2.resize(image, target_size) ## resize the image
            resized_images.append(resized_image) ## append the resized image to the list

    return np.array(resized_images) ## return the resized images as a numpy array
```

- A4) Resize the Image into (128,128) using the `resize_images_in_list` function

```
In [8]: resized_images = resize_images_in_list(raw_images)
```

- A5) Now plot again in the same figure one image from each class to verify that the B&W conversion and resizing has been succesfully performed.

```
In [9]: plt.figure(figsize=(7,7))
plt.suptitle('Resized Images',fontsize=15)

random_index_class0 = random.choice(np.where(labels == 0)[0])
plt.subplot(2, 2, 1)
plt.imshow(resized_images[random_index_class0],cmap='gray')
plt.title('Class 0', fontsize=10)

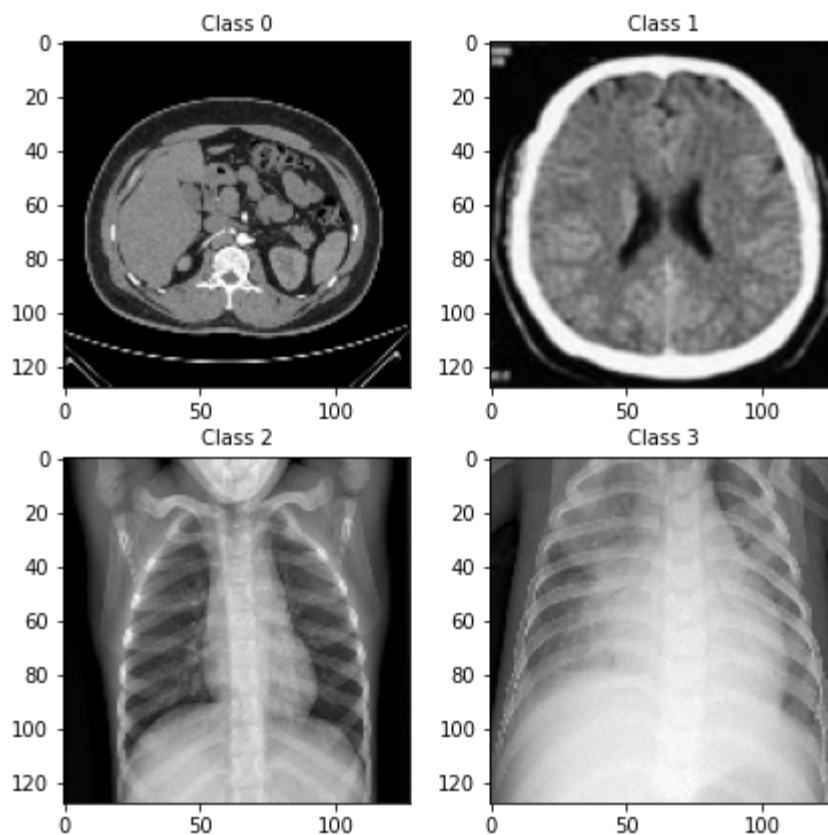
random_index_class1 = random.choice(np.where(labels == 1)[0])
plt.subplot(2, 2, 2)
plt.imshow(resized_images[random_index_class1],cmap='gray')
plt.title('Class 1', fontsize=10)

random_index_class2 = random.choice(np.where(labels == 2)[0])
plt.subplot(2, 2, 3)
plt.imshow(resized_images[random_index_class2],cmap='gray')
plt.title('Class 2', fontsize=10)

random_index_class3 = random.choice(np.where(labels == 3)[0])
plt.subplot(2, 2, 4)
plt.imshow(resized_images[random_index_class3],cmap='gray')
plt.title('Class 3', fontsize=10)
```

Out[9]: Text(0.5, 1.0, 'Class 3')

Resized Images



Task B - Kernels, Gradients and Convolution (3 points)

Edge detection is a technique used in image processing to identify the boundaries between different objects in an image. It works by detecting sharp changes in brightness or color within an image. These changes are called edges and can be used to segment an image into different regions. Edge detection is often used as a preprocessing step for other computer vision tasks, such as object detection and image segmentation.

```
In [10]: matrix = np.array([[0,0,0,0,0,0],[0,1,1,1,1,0],[0,1,1,1,1,0],[0,1,1,1,1,0],[0,1,1,1,1,0],[0,0,0,0,0,0]])

print("Matrix:")
print(matrix)

Matrix:
[[0 0 0 0 0 0]
 [0 1 1 1 1 0]
 [0 1 1 1 1 0]
 [0 1 1 1 1 0]
 [0 1 1 1 1 0]
 [0 0 0 0 0 0]]
```

Padding is a technique to increase the spatial dimensions of an input matrix by adding extra rows and columns of zeros around the edges of the input matrix. Padding can prevent loss of information as after a convolution operation the output matrix is smaller than the original one. It can also help to prevent the edges of the input matrix from being ignored by convolution operations. Manually perform the padding operation by adding a row and a column of zeros on all sides of the original matrix. Look at the given function called `manual_padding` that takes a matrix as an input and returns the padded matrix form.

```
In [11]: # Perform manual padding by adding a row and a column of zeros on all sides
def manual_padding(matrix):
    padded_matrix = np.zeros((matrix.shape[0] + 2, matrix.shape[1] + 2), dt
    ype=matrix.dtype)
    padded_matrix[1:-1, 1:-1] = matrix
    return padded_matrix
```

- B1) Using the given 4x4 matrix of integers to do a zero padding. Padding is made by adding a row and a column on all sides of the original matrix. Print the padded matrix.


```
In [12]: padded_matrix = manual_padding(matrix)

print("Padded Matrix:")
print(padded_matrix)
```

```
Padded Matrix:
[[0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 1 1 1 1 0 0]
 [0 0 1 1 1 1 0 0]
 [0 0 1 1 1 1 0 0]
 [0 0 1 1 1 1 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]]
```

Pooling operations are used to reduce the spatial dimensions of an input matrix while retaining the most important features. The most common types of pooling operations are max pooling and average pooling.

Max pooling takes the maximum value within a window of a given size and moves the window across the input matrix. This reduces the spatial dimensions of the input matrix while retaining the most important features.

Average pooling takes the average value within a window of a given size and moves the window across the input matrix. This also reduces the spatial dimensions of the input matrix while retaining the most important features.

Pooling operations can help to reduce the number of features used by a model, which can help to prevent overfitting and improve generalization.

Check the `max_pooling` function.

```
In [13]: def max_pooling(matrix):
          pool_size = 2
          pooled_matrix = np.zeros((matrix.shape[0] // pool_size + matrix.shape
[0] % 2, matrix.shape[1] // pool_size + matrix.shape[1] % 2), dtype=matrix.
dtype)
          for i in range(0, matrix.shape[0], pool_size):
              for j in range(0, matrix.shape[1], pool_size):
                  window = matrix[i:i+pool_size, j:j+pool_size]
                  max_value = np.max(window)
                  pooled_matrix[i//pool_size, j//pool_size] = max_value
          return pooled_matrix
```

- B2) Using the padded matrix you obtained in the last step, perform max pooling with a 2x2 window using the given “`max_pooling`”.

```
In [14]: max_pooled_matrix = max_pooling(padded_matrix)

print("Max Pooled Matrix:")
print(max_pooled_matrix)
```

```
Max Pooled Matrix:
[[0 0 0 0]
 [0 1 1 0]
 [0 1 1 0]
 [0 0 0 0]]
```

The code is defining two Sobel kernels for horizontal and vertical edge detection. These kernels are used in image processing to detect edges in an image. The horizontal Sobel kernel is used to detect horizontal edges, while the vertical Sobel kernel is used to detect vertical edges. These kernels are applied to an image using convolution to highlight the edges in the image.

```
In [15]: sobel_kernel_horizontal = np.array([[ -1,  0,  1],
                                             [ -2,  0,  2],
                                             [ -1,  0,  1]])

sobel_kernel_vertical = np.array([[ -1, -2, -1],
                                   [  0,  0,  0],
                                   [  1,  2,  1]])
```

The code defines a function called `convolution` that performs convolution between an input image and a given kernel.

```
In [16]: def convolution(image, kernel):
    # Get the dimensions of the image and kernel
    image_height, image_width = image.shape
    kernel_height, kernel_width = kernel.shape

    # Create an output image with the same dimensions as the input image
    output_image = np.zeros_like(image)

    # Pad the input image to handle boundary pixels
    padding = kernel_width // 2
    padded_image = np.pad(image, ((padding, padding), (padding, padding)),
mode='constant')

    # Perform convolution
    for i in range(image_height):
        for j in range(image_width):
            # Extract the region of interest (ROI) from the padded image
            roi = padded_image[i:i+kernel_height, j:j+kernel_width]

            # Compute the convolution result and store it in the output image
            output_image[i, j] = np.sum(roi * kernel)

    return np.array(output_image, dtype=np.float32)
```

- B3) Sobel filter is a commonly used filter in image processing. The purpose of using a sobel filter is to extract both the vertical and horizontal edges of the image. The fundamental operation of the sobel filter is convolution. Given both the vertical and horizontal Sobel kernel, Use the convolution function to apply the two kernels to detect the edges.
"gradient_x" is the result of convolution of sobel horizontal kernel while "gradient_y" is the result of the sobel vertical kernel.
- B4) Calculate the absolute value of the gradient and its orientation (HINT: Think of them as trigonometric components. np.arctan2() returns a value in radians, convert it to degrees.).

```
In [17]: gradient_x = convolution(max_pooled_matrix, sobel_kernel_horizontal)
gradient_y = convolution(max_pooled_matrix, sobel_kernel_vertical)
abs_gradient = np.sqrt(gradient_x**2 + gradient_y**2)
angle_gradient = np.arctan2(gradient_y, gradient_x)
```

- B5) Plot the original matrix and the results obtained in the step before in a single plot (1x5 subfigures) HINT: you can use any colormap for the gradients, for the angles plot use cmap='twilight' and plot the colorbar.

```

In [18]: plt.figure(figsize=(15,5))
plt.suptitle('Gradient Images',fontsize=15)

plt.subplot(1,5,1)
plt.imshow(max_pooled_matrix, cmap='gray')
plt.title('Original Matrix', fontsize=12)

plt.subplot(1, 5, 2)
plt.imshow(gradient_x, cmap='gray')
plt.title('Gradient X', fontsize=12)

plt.subplot(1, 5, 3)
plt.imshow(gradient_y, cmap='gray')
plt.title('Gradient Y', fontsize=12)

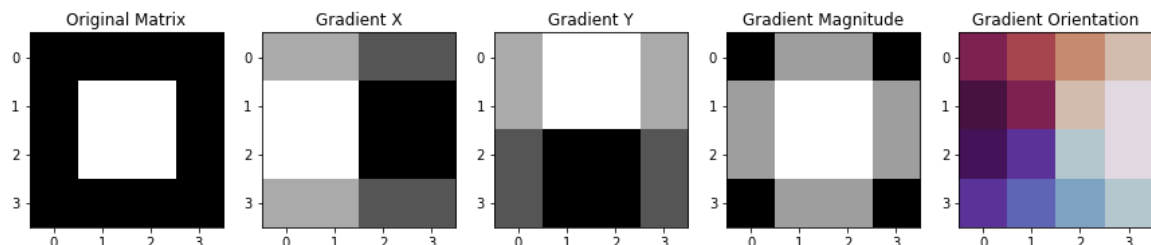
plt.subplot(1, 5, 4)
plt.imshow(abs_gradient, cmap='gray')
plt.title('Gradient Magnitude', fontsize=12)

plt.subplot(1, 5, 5)
plt.imshow(angle_gradient, cmap='twilight')
plt.title('Gradient Orientation', fontsize=12)

```

Out[18]: Text(0.5, 1.0, 'Gradient Orientation')

Gradient Images



Using the `mpimg.imread()` function, load the "airplane.tiff" image file and:

- B7) compute the horizontal and vertical gradients
- B8) compute the the magnitude of the gradient and its orientation
- B9) perform max pooling on the image and repeat the previous steps (gradients, magnitudes, orientations).

```
In [19]: image = mpimg.imread("airplane.tiff")

grad_x = convolution(image, sobel_kernel_horizontal)
grad_y = convolution(image, sobel_kernel_vertical)
abs_grad = np.sqrt(grad_x**2 + grad_y**2)
angle_grad = np.arctan2(grad_y, grad_x)

pooled_image = max_pooling(image)
pooled_grad_x = convolution(pooled_image, sobel_kernel_horizontal)
pooled_grad_y = convolution(pooled_image, sobel_kernel_vertical)
pooled_abs_grad = np.sqrt(pooled_grad_x**2 + pooled_grad_y**2)
pooled_angle_grad = np.arctan2(pooled_grad_y, pooled_grad_x)
```

- B10) plot all the results in a single plot containing all the above results (2x5 subfigures)

```
In [20]: plt.figure(figsize=(15,6))
plt.suptitle('Gradient Images',fontsize=15)

plt.subplot(2,5,1)
plt.imshow(image, cmap='gray')
plt.title('Original Matrix', fontsize=12)

plt.subplot(2, 5, 2)
plt.imshow(grad_x, cmap='gray')
plt.title('Gradient X', fontsize=12)

plt.subplot(2, 5, 3)
plt.imshow(grad_y, cmap='gray')
plt.title('Gradient Y', fontsize=12)

plt.subplot(2, 5, 4)
plt.imshow(abs_grad, cmap='gray')
plt.title('Gradient Magnitude', fontsize=12)

plt.subplot(2, 5, 5)
plt.imshow(angle_grad, cmap='twilight')
plt.title('Gradient Orientation', fontsize=12)

plt.subplot(2,5,6)
plt.imshow(pooled_image, cmap='gray')
plt.title('Pooled Matrix', fontsize=12)

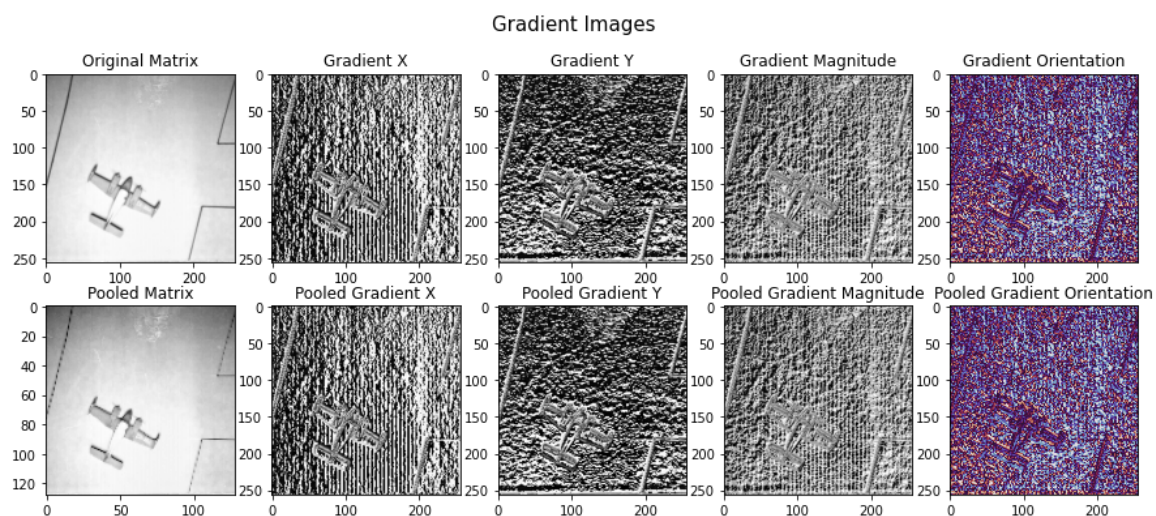
plt.subplot(2, 5, 7)
plt.imshow(pooled_grad_x, cmap='gray')
plt.title('Pooled Gradient X', fontsize=12)

plt.subplot(2, 5, 8)
plt.imshow(pooled_grad_y, cmap='gray')
plt.title('Pooled Gradient Y', fontsize=12)

plt.subplot(2, 5, 9)
plt.imshow(pooled_abs_grad, cmap='gray')
plt.title('Pooled Gradient Magnitude', fontsize=12)

plt.subplot(2, 5, 10)
plt.imshow(pooled_angle_grad, cmap='twilight')
plt.title('Pooled Gradient Orientation', fontsize=12)
```

```
Out[20]: Text(0.5, 1.0, 'Pooled Gradient Orientation')
```



Questions

1. What is the difference between convolution with a kernel and pooling? Hint: Look at the size of the kernels and the resulting images.
2. Why two different kernels are used for horizontal and vertical edge detection? Could a single kernel be used for both?
3. What differences can you spot between the pooled and the original image? and in the resulting gradients?
4. Although the image is very simple and our brains can infer the edges of the object very simply, why are the resulting edges very noisy? What could be implemented to mitigate the noise?

```
In [21]: print("Answer 1: Convolution focuses on feature extraction, while pooling f
          ocuses on reducing spatial dimensions")

          print("\nAnswer 2: Using separate kernels allows us to detect edges in diff
          erent directions, so horizontal direction and vertical direction. While a s
          ingle kernel could be used for both, it might not effectively capture edges
          at different angles.")

          print("\nAnswer 3: The pooled image is smaller than the original, resulting
          in a loss of detail. In the resulting gradients, there is a reduction in th
          e magnitude of changes compared to the original gradients.")

          print("\nAnswer 4: The resulting edges are noisy because the kernels used f
          or edge detection are sensitive to high-frequency noise in the image. To mi
          tigate noise, techniques like smoothing with a Gaussian filter.")
```

Answer 1: Convolution focuses on feature extraction, while pooling focuses on reducing spatial dimensions

Answer 2: Using separate kernels allows us to detect edges in different directions, so horizontal direction and vertical direction. While a single kernel could be used for both, it might not effectively capture edges at different angles.

Answer 3: The pooled image is smaller than the original, resulting in a loss of detail. In the resulting gradients, there is a reduction in the magnitude of changes compared to the original gradients.

Answer 4: The resulting edges are noisy because the kernels used for edge detection are sensitive to high-frequency noise in the image. To mitigate noise, techniques like smoothing with a Gaussian filter.

Task 3 - Feature Extraction: Histogram of Gradients and PCA (3 points)

In the context of feature extraction, HOG (Histogram of Oriented Gradients) and PCA (Principal Component Analysis) are two commonly used techniques.

Histogram of Oriented Gradients (HOG) is a feature extraction technique used to detect objects in images. It operates by dividing an image into small cells and then grouping the gradients in each cell according to a specified number of orientations. These grouped gradients, known as histograms, are concatenated to create a feature vector suitable for object detection. The HOG algorithm calculates gradient orientations in localized regions of an image, providing a comprehensive view of the image's gradient structure. To apply HOG, you can use the 'hog' function from the 'sklearn.feature' library, which allows you to extract both the HOG image and its corresponding HOG features. You can also print one of the processed images generated using the HOG technique.

Hint: The `get_hog` function is a function that calculates the Histogram of Oriented Gradients (HOG) features and images for a given set of images.


```
In [22]: from skimage.feature import hog

def get_hog(image_set, size_1, orientations=8):
    hog_images = []
    hog_features = []
    for image in image_set:
        fd, hog_image = hog(image, orientations=orientations, pixels_per_cell=(size_1, size_1), cells_per_block=(4,4), visualize=True)
        hog_images.append(hog_image)
        hog_features.append(fd)

    return np.array(hog_features), np.array(hog_images, dtype=np.float32)
```

- C1) Extract the HOG feature from the dataset, using size_1=[16,32] and orientations=[4, 8].

NOTE: computing the HOG features can take up to three minutes.

```
In [23]: #Not clear what image_set I should use, so I used the resized_images images set
hog_features_16_4, hog_images_16_4 = get_hog(resized_images, 16, 4)
hog_features_32_4, hog_images_32_4 = get_hog(resized_images, 32, 4)
hog_features_16_8, hog_images_16_8 = get_hog(resized_images, 16, 8)
hog_features_32_8, hog_images_32_8 = get_hog(resized_images, 32, 8)
```

- C2) Plot the 250th image of each variant of HOG images, and the shapes of the respective feature vectors.

```

In [24]: plt.figure(figsize=(7,7))
plt.suptitle('HOG Images',fontsize=15)

plt.subplot(2, 2, 1)
plt.imshow(hog_images_16_4[250])
plt.title('Size: 16, Orientations: 4', fontsize=12)

plt.subplot(2, 2, 2)
plt.imshow(hog_images_32_4[250])
plt.title('Size: 32, Orientations: 4', fontsize=12)

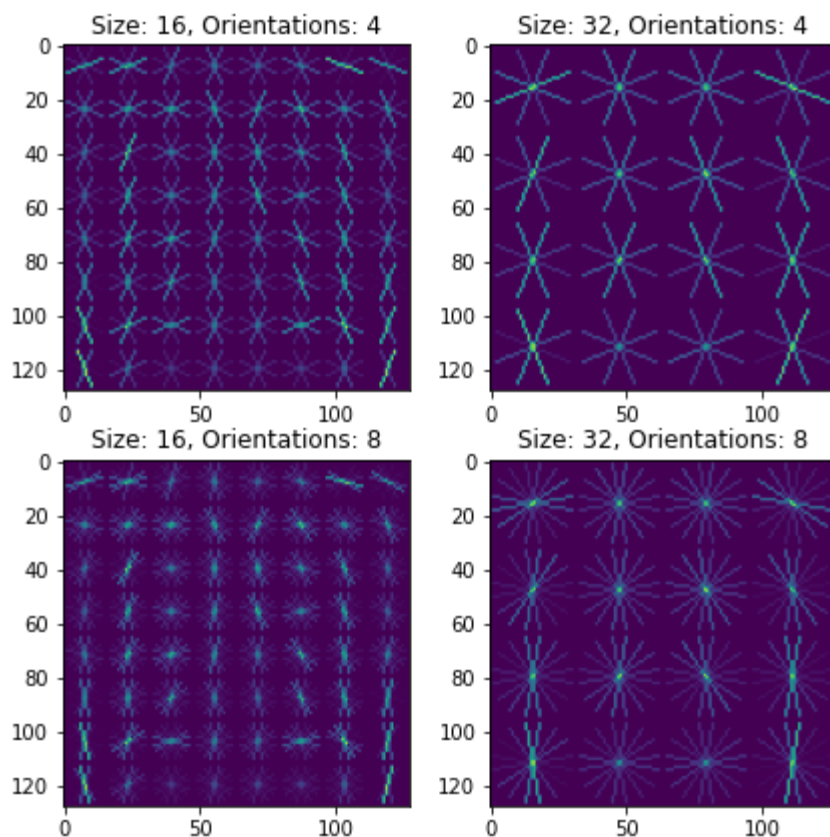
plt.subplot(2, 2, 3)
plt.imshow(hog_images_16_8[250])
plt.title('Size: 16, Orientations: 8', fontsize=12)

plt.subplot(2, 2, 4)
plt.imshow(hog_images_32_8[250])
plt.title('Size: 32, Orientations: 8', fontsize=12)

```

Out[24]: Text(0.5, 1.0, 'Size: 32, Orientations: 8')

HOG Images



```
In [25]: print("Shapes of feauter vector Size: 16, Orientations: 4 " + str(hog_featu
res_16_4[250].shape))
print("Shapes of feauter vector Size: 32, Orientations: 4 " + str(hog_featu
res_32_4[250].shape))
print("Shapes of feauter vector Size: 16, Orientations: 8 " + str(hog_featu
res_16_8[250].shape))
print("Shapes of feauter vector Size: 32, Orientations: 8 " + str(hog_featu
res_32_8[250].shape))

Shapes of feauter vector Size: 16, Orientations: 4 (1600,)
Shapes of feauter vector Size: 32, Orientations: 4 (64,)
Shapes of feauter vector Size: 16, Orientations: 8 (3200,)
Shapes of feauter vector Size: 32, Orientations: 8 (128,)
```

Principal Component Analysis (PCA) is a fundamental method in machine learning used for constructing features and dimensionality reduction. In this section, you will utilize the PCA function from 'sklearn.decomposition' to extract features from the provided dataset. PCA identifies the principal components of a dataset, which represent the directions in which the data exhibits the most variation. By projecting a feature vector onto a lower-dimensional subspace defined by these principal components, PCA effectively reduces the dimensionality of the data while retaining the essential information capturing the primary data variations.

- C3) Apply PCA to the resized dataset. Hint: PCA works in one dimension only, so reshape the images accordingly.
- C4) Plot (scatter) the first two components of the PCA transformed features.

```
In [26]: from sklearn.decomposition import PCA

x_flattened = [image.flatten() for image in resized_images]

pca = PCA(n_components=2)
x_pca = pca.fit_transform(x_flattened)

# Plot the first two principal components of each point
plt.figure(figsize=(5,5))
unique_labels = list(set(labels))
for label in unique_labels:
    plt.scatter(x_pca[labels == label, 0], x_pca[labels == label, 1], label
=f'Class {label}', cmap="brg")
plt.xlabel("First component")
plt.ylabel("Second component")
plt.title("PCA of Flattened Images")
plt.legend()
plt.show()
```



- C5) Plot the explained variance ratio and cumulative explained variance ratio of the first 20 components.

```

In [27]: pca_20 = PCA(n_components=20)
x_pca_20 = pca_20.fit_transform(x_flattened)

explained_variance_ratio = pca_20.explained_variance_ratio_
cumulative_explained_variance_ratio = np.cumsum(explained_variance_ratio)

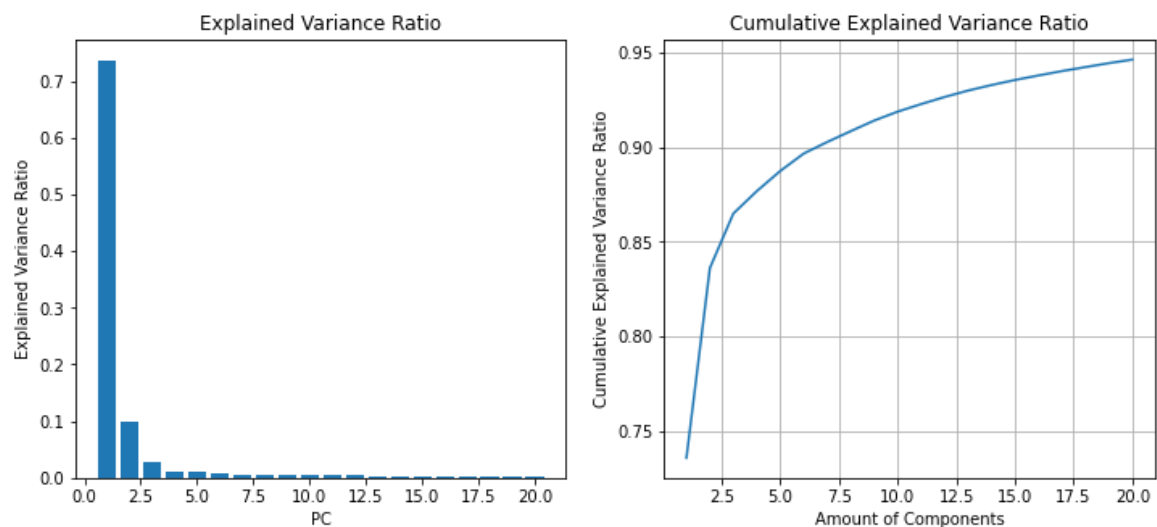
plt.figure(figsize=(5,5))

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.bar(range(1, 21), explained_variance_ratio)
plt.xlabel('PC')
plt.ylabel('Explained Variance Ratio')
plt.title('Explained Variance Ratio')

plt.subplot(1, 2, 2)
plt.plot(range(1, 21), cumulative_explained_variance_ratio)
plt.xlabel('Amount of Components')
plt.ylabel('Cumulative Explained Variance Ratio')
plt.title('Cumulative Explained Variance Ratio')
plt.grid(True)
plt.show()

```

<Figure size 360x360 with 0 Axes>



```
In [28]: print("Answer 1: HOG images extracted using 16 pixels/cell have smaller cell sizes and result in larger feature vectors compared to HOG images extracted using 32 pixels/cell. The original flattened images have the largest feature vectors.")

print("\nAnswer 2: Model performance varying between the generated HOG images affect the classification results")

print("\nAnswer 3: We see that data points from different classes exhibit separation, so that there is variance between the classes.")

print("\nAnswer 4: Classes 2 and 3 show that they have some overlap in their feature space, making it challenging to separate them based on the first two components.")

print("\nAnswer 5: The explained variance of each component represents the proportion of the total variance in the data that is explained by that specific principal component. It indicates how much information is retained by each component after reducing the dimension of the data")
```

Answer 1: HOG images extracted using 16 pixels/cell have smaller cell sizes and result in larger feature vectors compared to HOG images extracted using 32 pixels/cell. The original flattened images have the largest feature vectors.

Answer 2: Model performance varying between the generated HOG images affect the classification results

Answer 3: We see that data points from different classes exhibit separation, so that there is variance between the classes.

Answer 4: Classes 2 and 3 show that they have some overlap in their feature space, making it challenging to separate them based on the first two components.

Answer 5: The explained variance of each component represents the proportion of the total variance in the data that is explained by that specific principal component. It indicates how much information is retained by each component after reducing the dimension of the data

Questions:

1. What did you notice between the HOG images extracted using 16 or 32 pixels/cell? Relate to feature vector(s) sizes and compare it with the original flattened images.
2. How does the model performance vary between generated HOG images?
3. In the PCA plot, what do you observe in the two-component plot?
4. Can you relate the description of Classes 2 and 3 in the dataset, with the behavior highlighted by the previous question?
5. What does the explained variance of each component represent?

Task D - Model Training (3 points)

In this task you are going to classify images and the related extracted features . We will define a function “Classifier” that fits the classification model, compute Accuracy, Recall, Precision, F1, and AUC and plot the confusion matrix.

```
In [29]: from sklearn.model_selection import train_test_split
         from sklearn.neighbors import KNeighborsClassifier
         from sklearn.svm import SVC
         from sklearn.neural_network import MLPClassifier
         from sklearn.metrics import accuracy_score, f1_score, precision_score, roc_auc_score, recall_score, confusion_matrix
         import seaborn as sns
```

The code defines a function called Classifier that takes as input parameters: clf (a classifier object), x_train (the training data features), y_train (the training data labels), x_test (the test data features), and y_test (the test data labels).

- D1) Complete the Classifier function so that:
 - The input model is fitted to the given data, and predictions are made on the test data.
 - The following metrics are computed with respect to the test data and printed:
 - Accuracy
 - Precision
 - Recall
 - F1 Score.

Note: for these last 3 metrics, when computing include the argument average='macro'.

- The confusion matrix is plotted and displayed.

```
In [30]: def Classifier(clf,x_train,y_train,x_test,y_test):

    #Fit the model
    clf.fit(x_train, y_train)

    # Predict on the test set
    y_pred = clf.predict(x_test)

    # Print the scores
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='macro')
    recall = recall_score(y_test, y_pred, average='macro')
    f1 = f1_score(y_test, y_pred, average='macro')

    print(f"Accuracy: {accuracy:.2f}")
    print(f"Precision: {precision:.2f}")
    print(f"Recall: {recall:.2f}")
    print(f"F1 Score: {f1:.2f}")

    # Plot the confusion matrix
    plt.figure(figsize=(5, 5))
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
    plt.xlabel("Predicted Labels")
    plt.ylabel("True Labels")
    plt.title("Confusion Matrix")
    plt.show()
```

From now on, when creating train/test splits, use `test_size=0.2`, `random_state=42`

- D2) train the following models on the flattened image data:
 - k-NN, using as `n_neighbors=[3,4,5]`
 - SVM, changing between `kernels=['linear', 'poly', 'rbf']`, when using poly use `degree=3`.
For all kernels use `[gamma='auto', random_state=42, C=1.0, coef0=0.0, tol=1e-3]`.
 - MLP, changing the `hidden_layer_sizes=[(100),(100,100),(100,100,100)]`.
For the remaining parameters always use `[activation='relu', solver='adam', alpha=0.0001, max_iter=200, shuffle=True, random_state=42]`

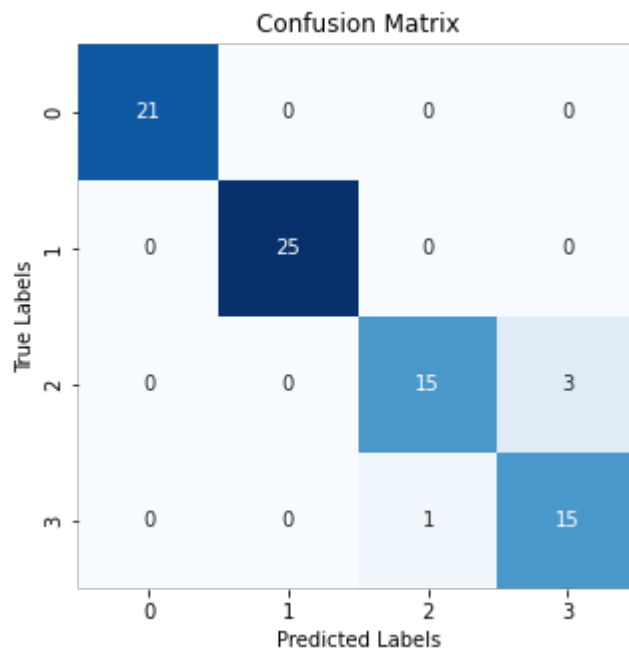

```
In [31]: x_train, x_test, y_train, y_test = train_test_split(x_flattened, labels, test_size=0.2, random_state=42)

### knn
n_neighbors=[3,4,5]
for i in n_neighbors:
    knn_model = KNeighborsClassifier(n_neighbors=i)
    Classifier(knn_model,x_train,y_train,x_test,y_test)

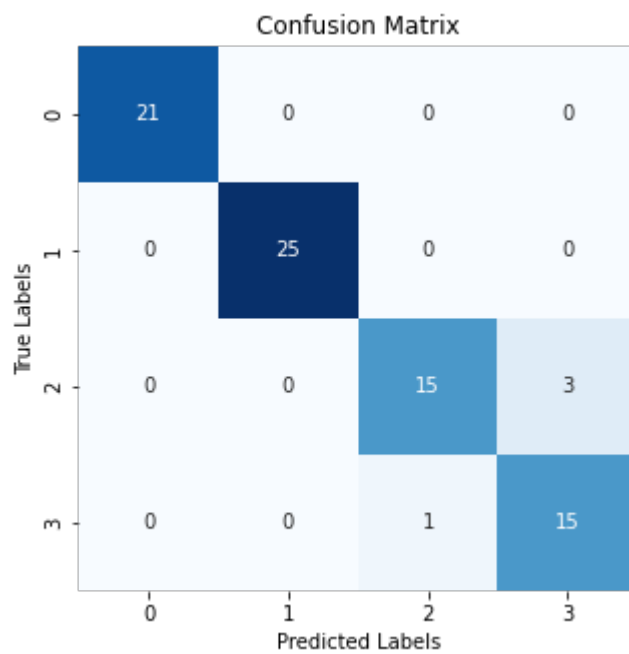
### svc
kernels=['linear','poly','rbf']
for j in kernels:
    svc_model = SVC(kernel=j, gamma='auto', random_state=42, C=1.0, coef0=0.0, tol=1e-3)
    Classifier(svc_model,x_train,y_train,x_test,y_test)

### mlp
hidden_layer_sizes=[(100),(100,100),(100,100,100)]
for k in hidden_layer_sizes:
    mlp_model = MLPClassifier(hidden_layer_sizes=k, activation='relu', solver='adam', alpha=0.0001, max_iter=200, shuffle=True, random_state=42)
    Classifier(mlp_model,x_train,y_train,x_test,y_test)
```

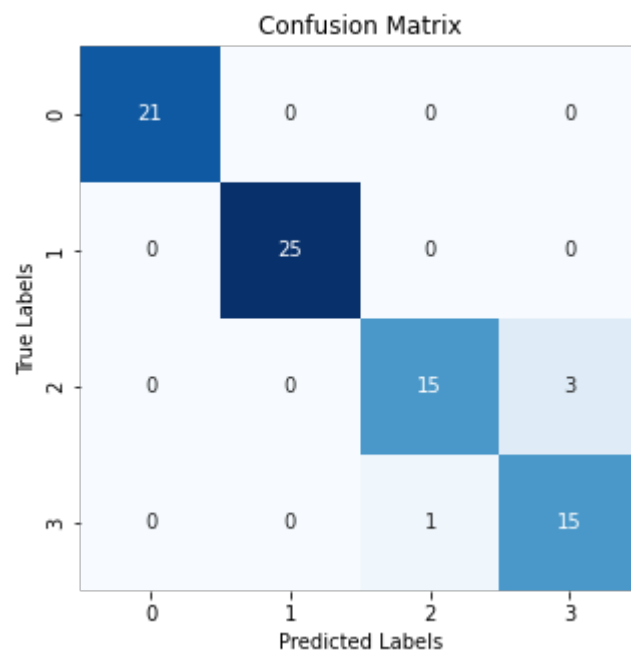
Accuracy: 0.95
Precision: 0.94
Recall: 0.94
F1 Score: 0.94



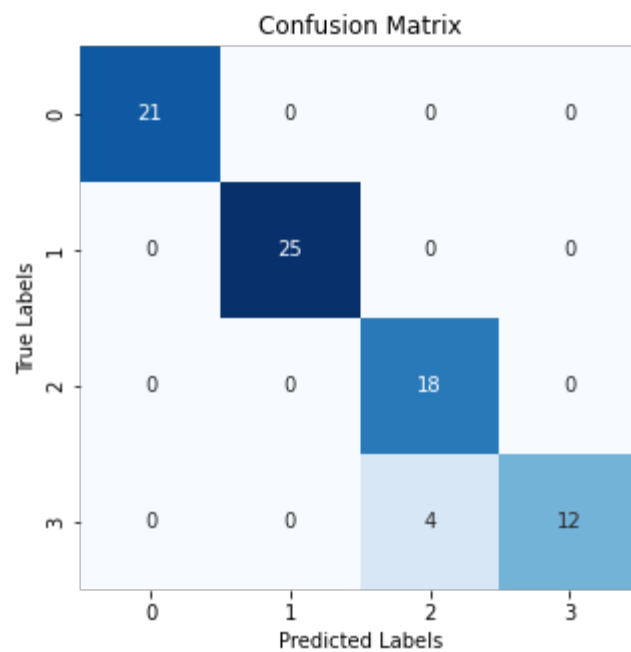
Accuracy: 0.95
Precision: 0.94
Recall: 0.94
F1 Score: 0.94



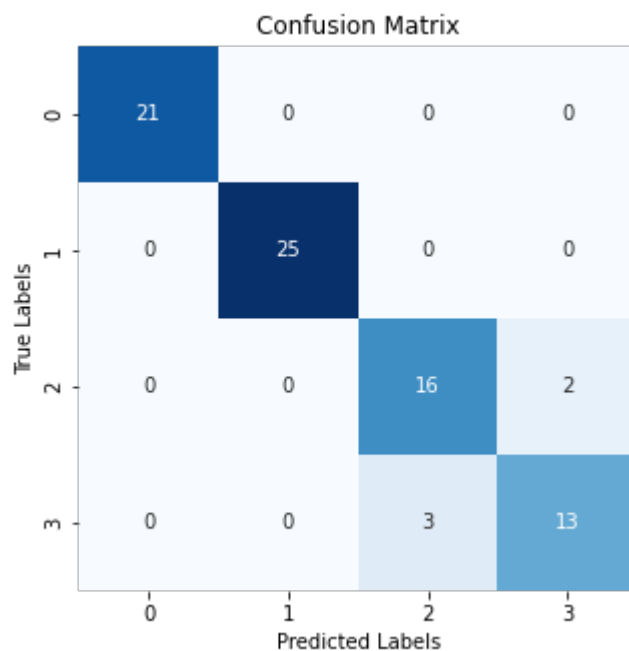
Accuracy: 0.95
Precision: 0.94
Recall: 0.94
F1 Score: 0.94



Accuracy: 0.95
Precision: 0.95
Recall: 0.94
F1 Score: 0.94



Accuracy: 0.94
Precision: 0.93
Recall: 0.93
F1 Score: 0.93



/usr/lib/python3.7/site-packages/sklearn/metrics/_classification.py:1308: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

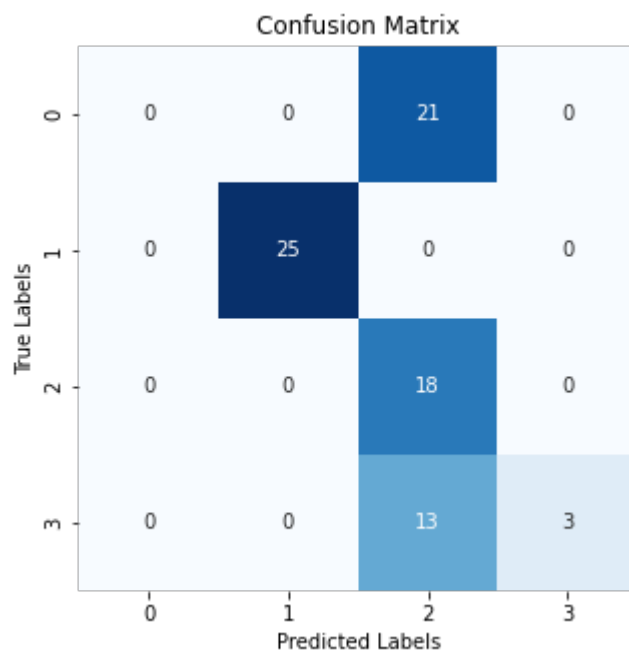
_warn_prf(average, modifier, msg_start, len(result))

Accuracy: 0.57

Precision: 0.59

Recall: 0.55

F1 Score: 0.46

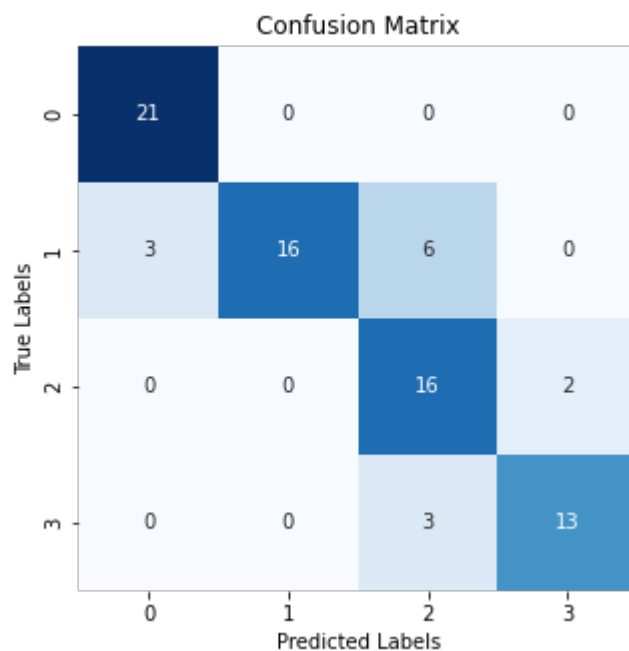


Accuracy: 0.82

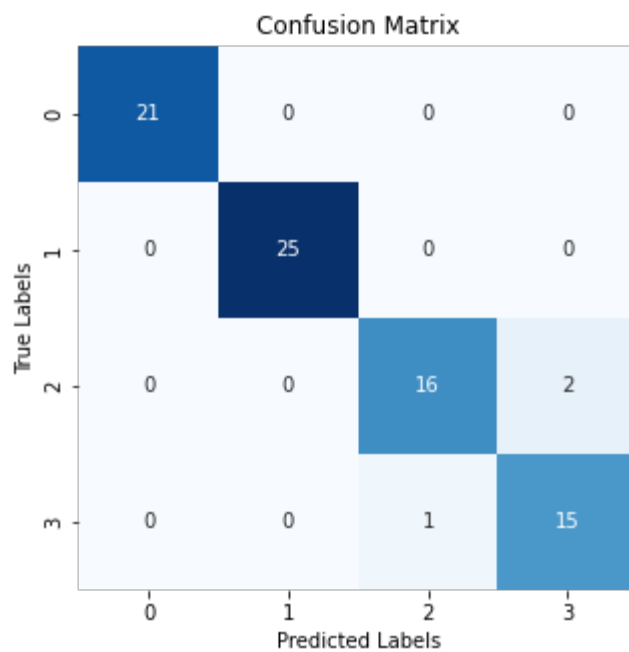
Precision: 0.85

Recall: 0.84

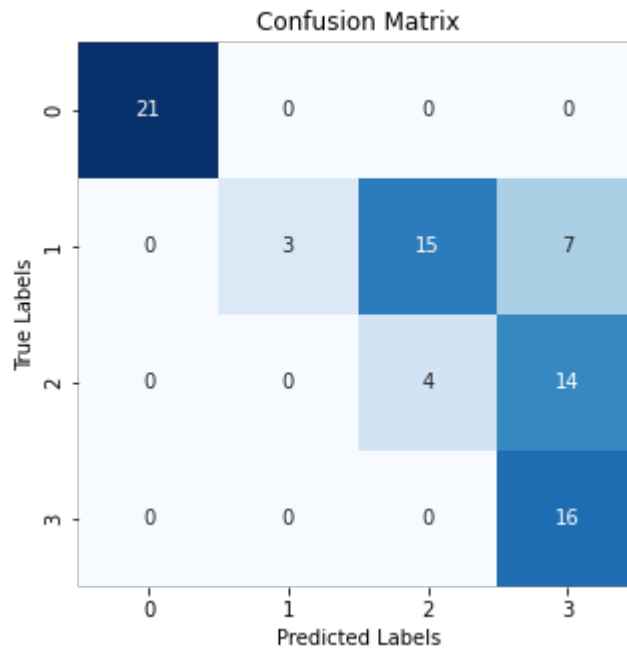
F1 Score: 0.82



Accuracy: 0.96
Precision: 0.96
Recall: 0.96
F1 Score: 0.96



Accuracy: 0.55
Precision: 0.66
Recall: 0.59
F1 Score: 0.51



- D3) Using the 4 sets of HOG features obtained before, repeat the training process on new classifier instances, this time use:
 - kNN, with neighbors=4
 - SVC, with kernel='linear'
Again use [gamma='auto', random_state=42, C=1.0, coef0=0.0, tol=1e-3].
 - MLP, with hidden_layer_size=(100)
For the remaining parameters always use [activation='relu', solver='adam', alpha=0.0001, max_iter=200, shuffle=True, random_state=42]

```
In [32]: x_train_hog_16_4, x_test_hog_16_4, y_train_hog_16_4, y_test_hog_16_4 = train_test_split(hog_features_16_4, labels, test_size=0.2, random_state=42)
x_train_hog_32_4, x_test_hog_32_4, y_train_hog_32_4, y_test_hog_32_4 = train_test_split(hog_features_32_4, labels, test_size=0.2, random_state=42)
x_train_hog_16_8, x_test_hog_16_8, y_train_hog_16_8, y_test_hog_16_8 = train_test_split(hog_features_16_8, labels, test_size=0.2, random_state=42)
x_train_hog_32_8, x_test_hog_32_8, y_train_hog_32_8, y_test_hog_32_8 = train_test_split(hog_features_32_8, labels, test_size=0.2, random_state=42)

knn_hog_16_4 = KNeighborsClassifier(n_neighbors=4)
Classifier(knn_hog_16_4, x_train_hog_16_4, y_train_hog_16_4, x_test_hog_16_4, y_test_hog_16_4)

knn_hog_32_4 = KNeighborsClassifier(n_neighbors=4)
Classifier(knn_hog_32_4, x_train_hog_32_4, y_train_hog_32_4, x_test_hog_32_4, y_test_hog_32_4)

knn_hog_16_8 = KNeighborsClassifier(n_neighbors=4)
Classifier(knn_hog_16_8, x_train_hog_16_8, y_train_hog_16_8, x_test_hog_16_8, y_test_hog_16_8)

knn_hog_32_8 = KNeighborsClassifier(n_neighbors=4)
Classifier(knn_hog_32_8, x_train_hog_32_8, y_train_hog_32_8, x_test_hog_32_8, y_test_hog_32_8)

### svc

svc_hog_16_4 = SVC(kernel='linear', gamma='auto', random_state=42, C=1.0, coef0=0.0, tol=1e-3)
Classifier(svc_hog_16_4, x_train_hog_16_4, y_train_hog_16_4, x_test_hog_16_4, y_test_hog_16_4)

svc_hog_32_4 = SVC(kernel='linear', gamma='auto', random_state=42, C=1.0, coef0=0.0, tol=1e-3)
Classifier(svc_hog_32_4, x_train_hog_32_4, y_train_hog_32_4, x_test_hog_32_4, y_test_hog_32_4)

svc_hog_16_8 = SVC(kernel='linear', gamma='auto', random_state=42, C=1.0, coef0=0.0, tol=1e-3)
Classifier(svc_hog_16_8, x_train_hog_16_8, y_train_hog_16_8, x_test_hog_16_8, y_test_hog_16_8)

svc_hog_32_8 = SVC(kernel='linear', gamma='auto', random_state=42, C=1.0, coef0=0.0, tol=1e-3)
Classifier(svc_hog_32_8, x_train_hog_32_8, y_train_hog_32_8, x_test_hog_32_8, y_test_hog_32_8)

### mlp

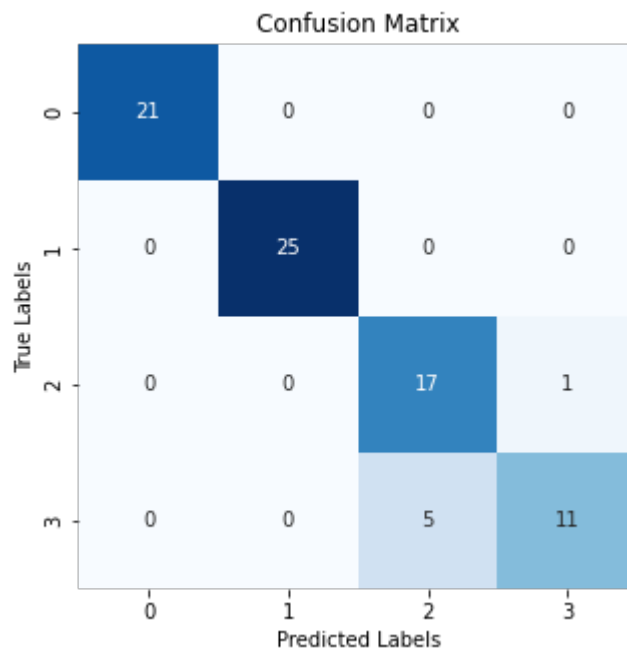
mlp_hog_16_4 = MLPClassifier(hidden_layer_sizes=100, activation='relu', solver='adam', alpha=0.0001, max_iter=200, shuffle=True, random_state=42)
Classifier(mlp_hog_16_4, x_train_hog_16_4, y_train_hog_16_4, x_test_hog_16_4, y_test_hog_16_4)
```

```
mlp_hog_32_4 = MLPClassifier(hidden_layer_sizes=100 ,activation='relu', solver='adam', alpha=0.0001, max_iter=200, shuffle=True, random_state=42)
Classifier(mlp_hog_32_4, x_train_hog_32_4, y_train_hog_32_4, x_test_hog_32_4, y_test_hog_32_4)

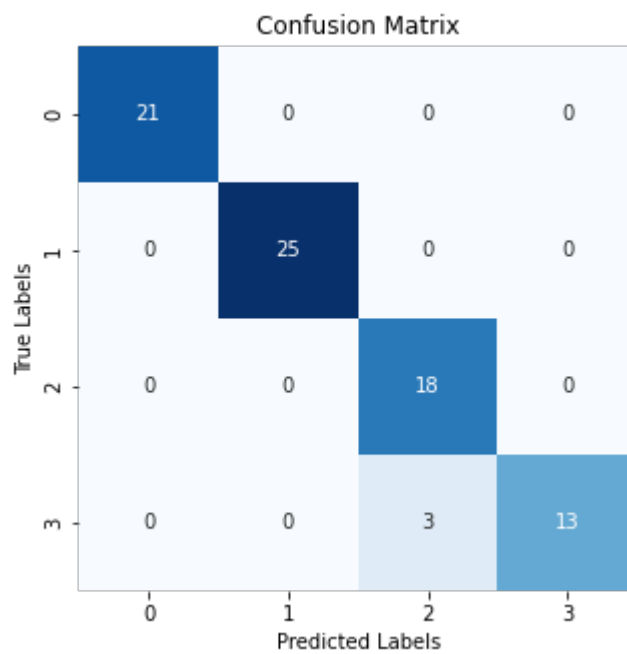
mlp_hog_16_8 = MLPClassifier(hidden_layer_sizes=100 ,activation='relu', solver='adam', alpha=0.0001, max_iter=200, shuffle=True, random_state=42)
Classifier(mlp_hog_16_8, x_train_hog_16_8, y_train_hog_16_8, x_test_hog_16_8, y_test_hog_16_8)

mlp_hog_32_8 = MLPClassifier(hidden_layer_sizes=100 ,activation='relu', solver='adam', alpha=0.0001, max_iter=200, shuffle=True, random_state=42)
Classifier(mlp_hog_32_8, x_train_hog_32_8, y_train_hog_32_8, x_test_hog_32_8, y_test_hog_32_8)
```

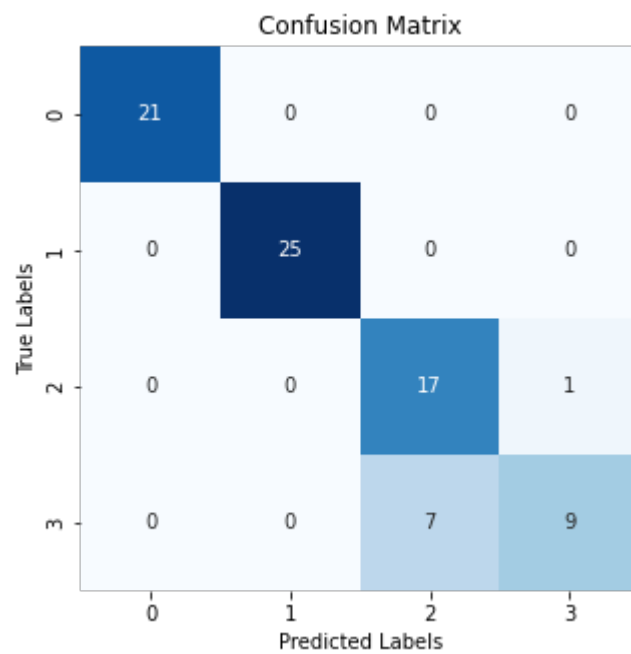

Accuracy: 0.93
Precision: 0.92
Recall: 0.91
F1 Score: 0.91



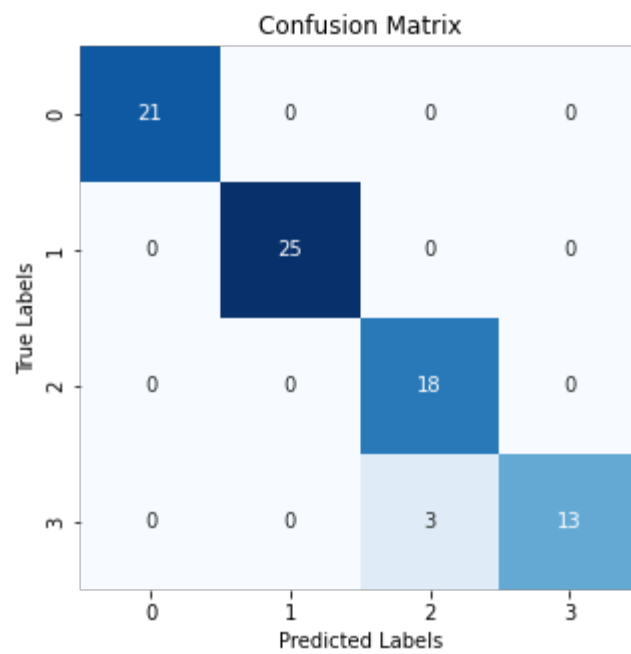
Accuracy: 0.96
Precision: 0.96
Recall: 0.95
F1 Score: 0.95



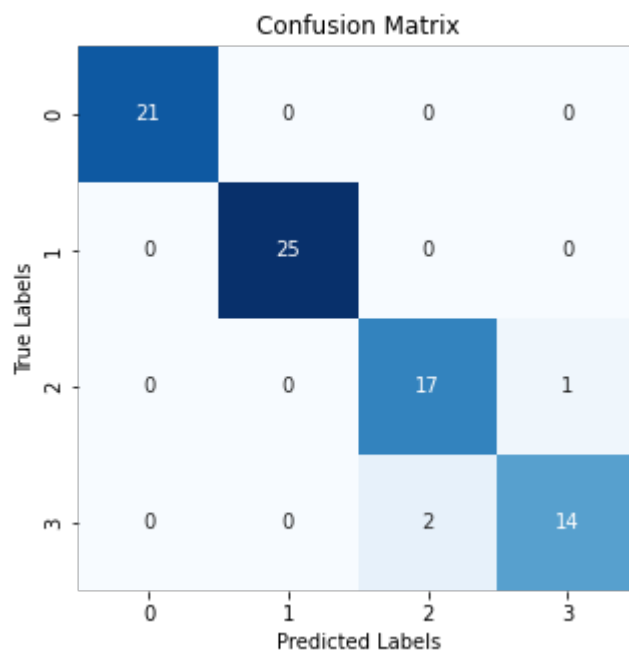
Accuracy: 0.90
Precision: 0.90
Recall: 0.88
F1 Score: 0.88



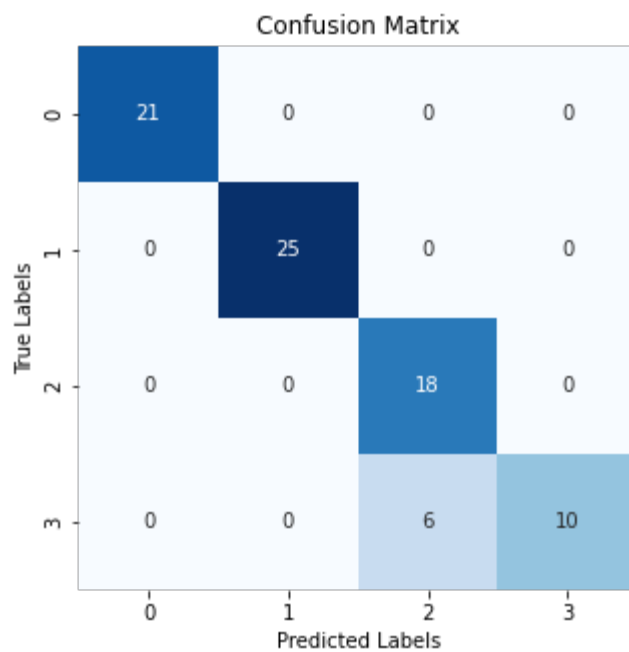
Accuracy: 0.96
Precision: 0.96
Recall: 0.95
F1 Score: 0.95



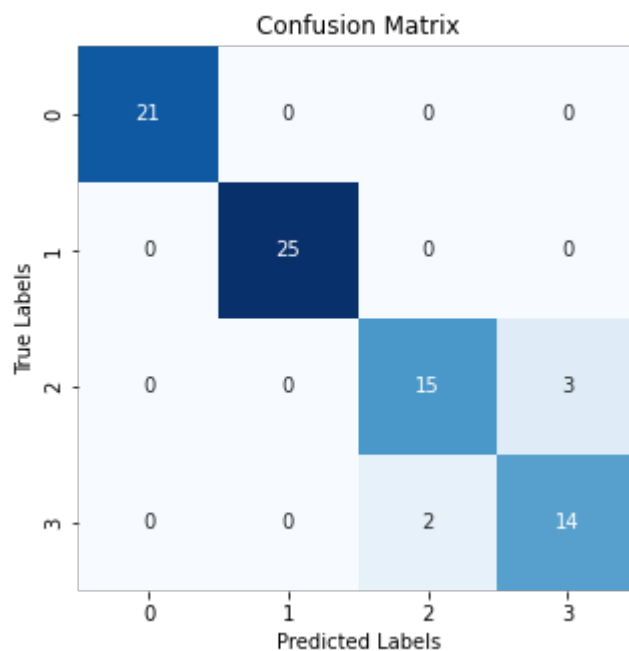
Accuracy: 0.96
Precision: 0.96
Recall: 0.95
F1 Score: 0.96



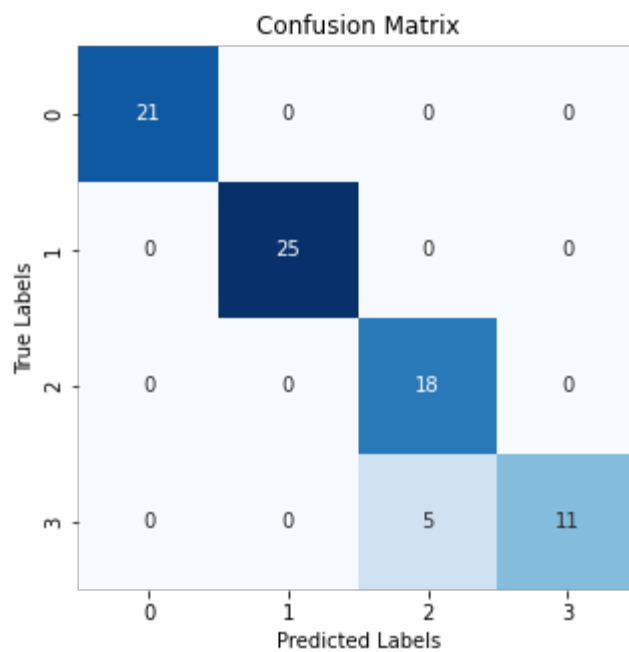
Accuracy: 0.93
Precision: 0.94
Recall: 0.91
F1 Score: 0.91



Accuracy: 0.94
Precision: 0.93
Recall: 0.93
F1 Score: 0.93

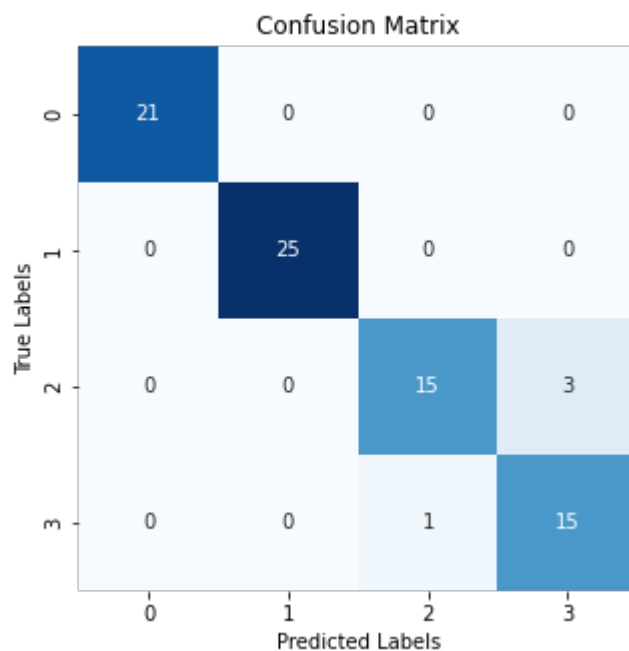


Accuracy: 0.94
 Precision: 0.95
 Recall: 0.92
 F1 Score: 0.92



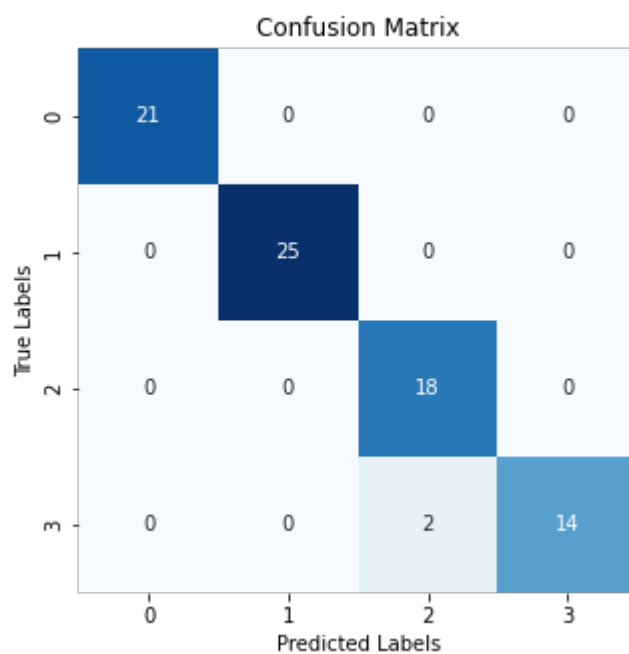
Accuracy: 0.95
 Precision: 0.94
 Recall: 0.94
 F1 Score: 0.94

/usr/lib/python3.7/site-packages/sklearn/neural_network/_multilayer_perceptron.py:696: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
 ConvergenceWarning,

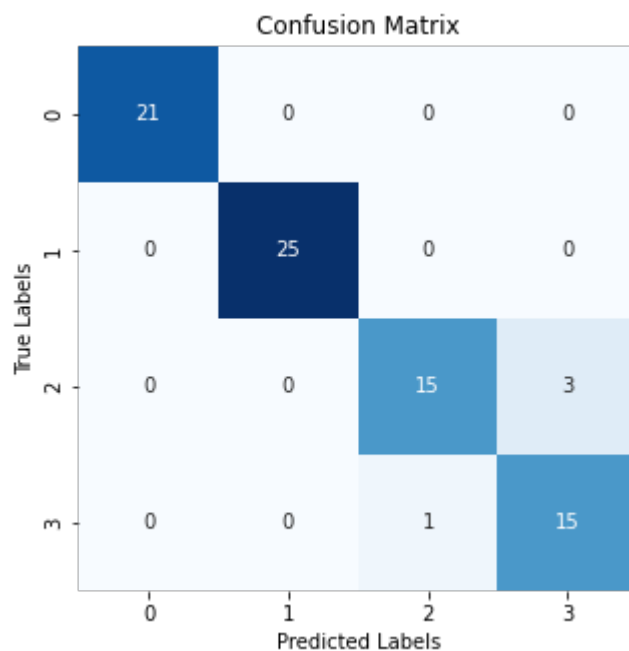


Accuracy: 0.97
Precision: 0.97
Recall: 0.97
F1 Score: 0.97

/usr/lib/python3.7/site-packages/sklearn/neural_network/_multilayer_perceptron.py:696: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
ConvergenceWarning,

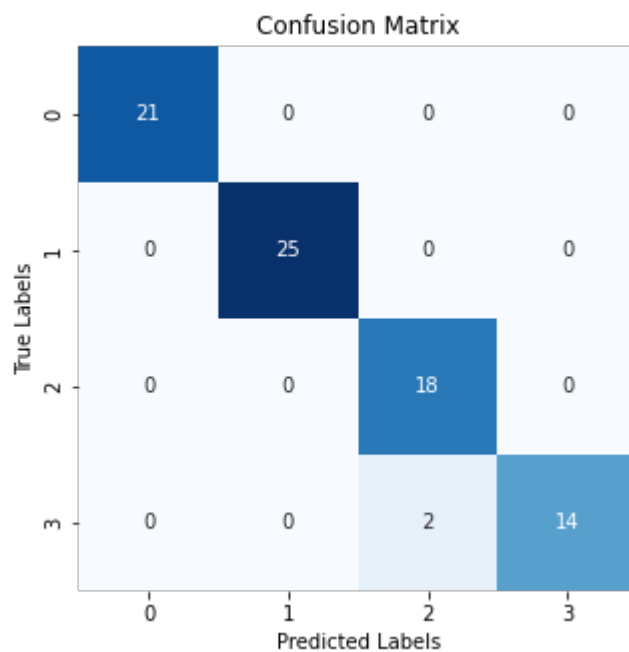


Accuracy: 0.95
Precision: 0.94
Recall: 0.94
F1 Score: 0.94



Accuracy: 0.97
Precision: 0.97
Recall: 0.97
F1 Score: 0.97

/usr/lib/python3.7/site-packages/sklearn/neural_network/_multilayer_perceptron.py:696: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
ConvergenceWarning,



- D4) Now repeat the training step using the PCA features, using:
 - kNN, with neighbors=4
 - SVC, with kernel='linear'
 - MLP, with hidden_layer_size=(100)
again use for the remaining parameters always use [activation='relu',
solver='adam', alpha=0.0001, max_iter=200, shuffle=True, random_state=42]

Train all models first using only the first two PCA components, then the first 10, then all.

```
In [35]: pca_2 = PCA(n_components=2)
x_pca_2 = pca_2.fit_transform(x_flattened)

pca_10 = PCA(n_components=10)
x_pca_10 = pca_10.fit_transform(x_flattened)

pca_all = PCA(n_components=len(labels))
x_pca_all = pca_all.fit_transform(x_flattened)

x_train_pca_2, x_test_pca_2, y_train_pca_2, y_test_pca_2 = train_test_split(
    x_pca_2, labels, test_size=0.2, random_state=42)
x_train_pca_10, x_test_pca_10, y_train_pca_10, y_test_pca_10 = train_test_s
plit(x_pca_10, labels, test_size=0.2, random_state=42)
x_train_pca_all, x_test_pca_all, y_train_pca_all, y_test_pca_all = train_te
st_split(x_pca_10, labels, test_size=0.2, random_state=42)

### knn and pca with 2 components
knn_pca_2 = KNeighborsClassifier(n_neighbors=4)
Classifier(knn_pca_2, x_train_pca_2, y_train_pca_2, x_test_pca_2, y_test_pc
a_2)

### knn and pca with 10 components
knn_pca_10 = KNeighborsClassifier(n_neighbors=4)
Classifier(knn_pca_10, x_train_pca_10, y_train_pca_10, x_test_pca_10, y_tes
t_pca_10)

### knn and pca with all components
knn_pca_all = KNeighborsClassifier(n_neighbors=4)
Classifier(knn_pca_all, x_train_pca_all, y_train_pca_all, x_test_pca_all, y
_test_pca_all)

### svc and pca with 2 components

svc_pca_2 = SVC(kernel='linear', gamma='auto', random_state=42, C=1.0, coef
f0=0.0, tol=1e-3)
Classifier(svc_pca_2, x_train_pca_2, y_train_pca_2, x_test_pca_2, y_test_pc
a_2)

svc_pca_10 = SVC(kernel='linear', gamma='auto', random_state=42, C=1.0, coe
f0=0.0, tol=1e-3)
Classifier(svc_pca_10, x_train_pca_10, y_train_pca_10, x_test_pca_10, y_tes
t_pca_10)

svc_pca_all = SVC(kernel='linear', gamma='auto', random_state=42, C=1.0, co
ef0=0.0, tol=1e-3)
Classifier(svc_pca_all, x_train_pca_all, y_train_pca_all, x_test_pca_all, y
_test_pca_all)

### mlp and pca with 2 components

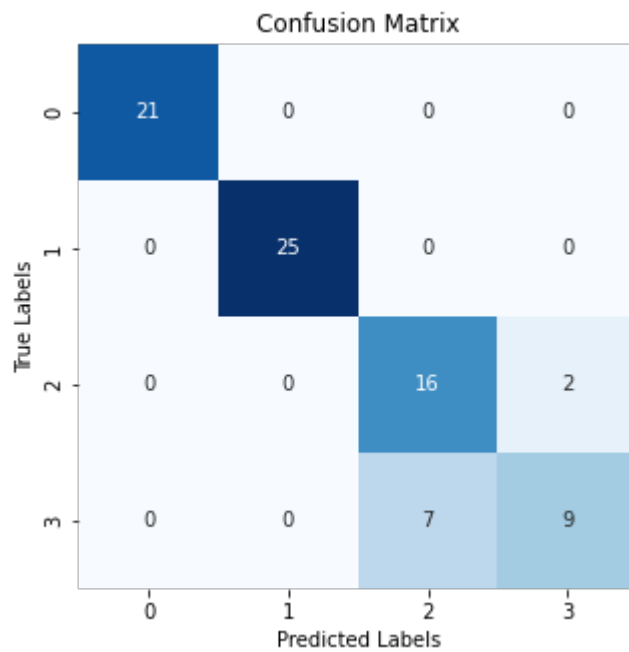
mlp_pca_2 = MLPClassifier(hidden_layer_sizes=100, activation='relu', solver
='adam', alpha=0.0001, max_iter=200, shuffle=True, random_state=42)
Classifier(mlp_pca_2, x_train_pca_2, y_train_pca_2, x_test_pca_2, y_test_pc
a_2)
```



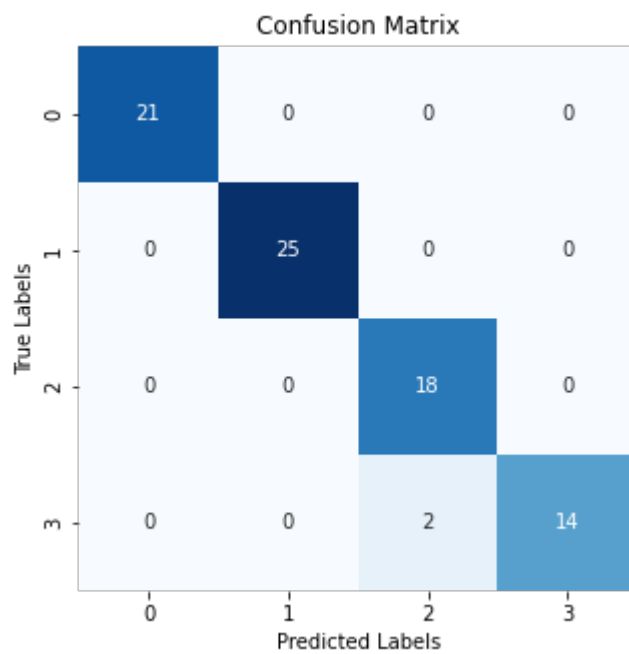
```
mlp_pca_10 = MLPClassifier(hidden_layer_sizes=100 ,activation='relu', solver='adam', alpha=0.0001, max_iter=200, shuffle=True, random_state=42)
Classifier(mlp_pca_10, x_train_pca_10, y_train_pca_10, x_test_pca_10, y_test_pca_10)

mlp_pca_all = MLPClassifier(hidden_layer_sizes=100 ,activation='relu', solver='adam', alpha=0.0001, max_iter=200, shuffle=True, random_state=42)
Classifier(mlp_pca_all, x_train_pca_all, y_train_pca_all, x_test_pca_all, y_test_pca_all)
```

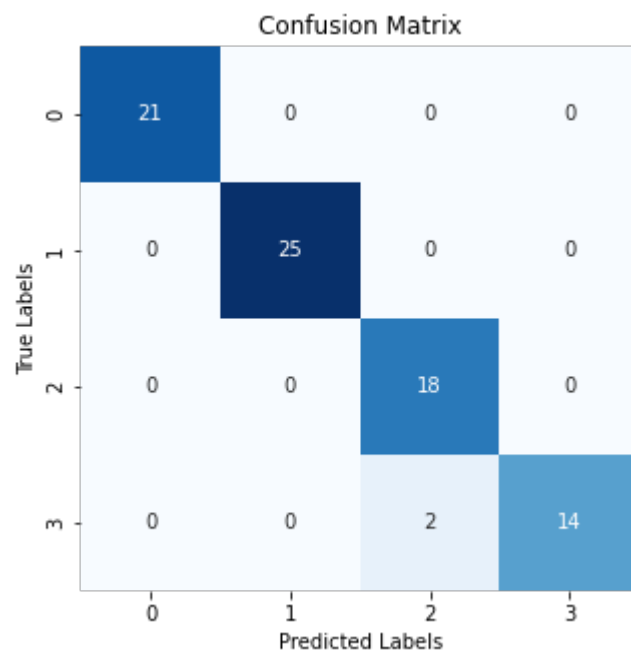
Accuracy: 0.89
Precision: 0.88
Recall: 0.86
F1 Score: 0.86



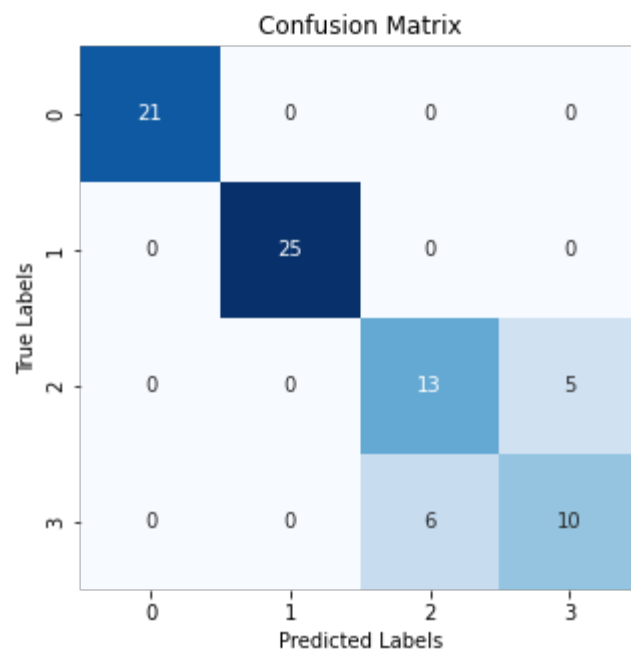
Accuracy: 0.97
Precision: 0.97
Recall: 0.97
F1 Score: 0.97



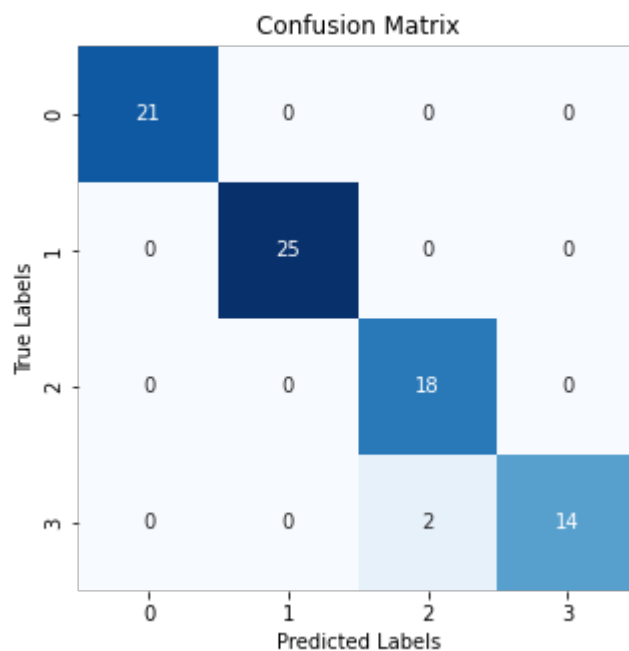
Accuracy: 0.97
Precision: 0.97
Recall: 0.97
F1 Score: 0.97



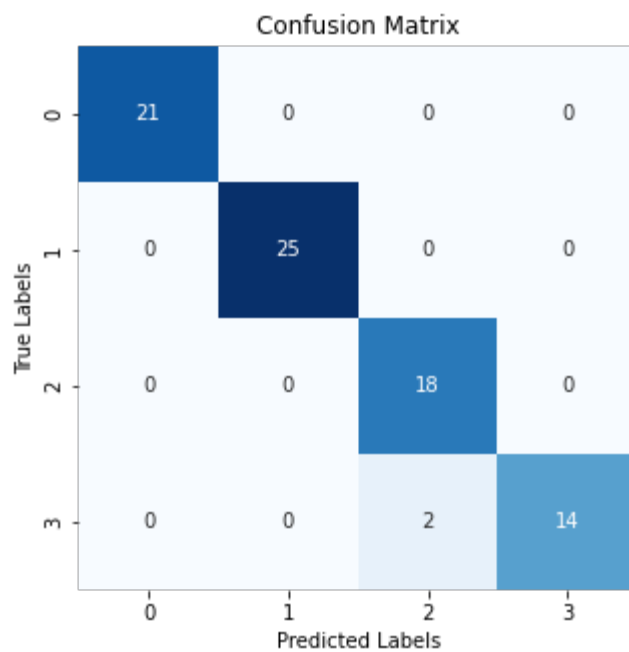
Accuracy: 0.86
Precision: 0.84
Recall: 0.84
F1 Score: 0.84



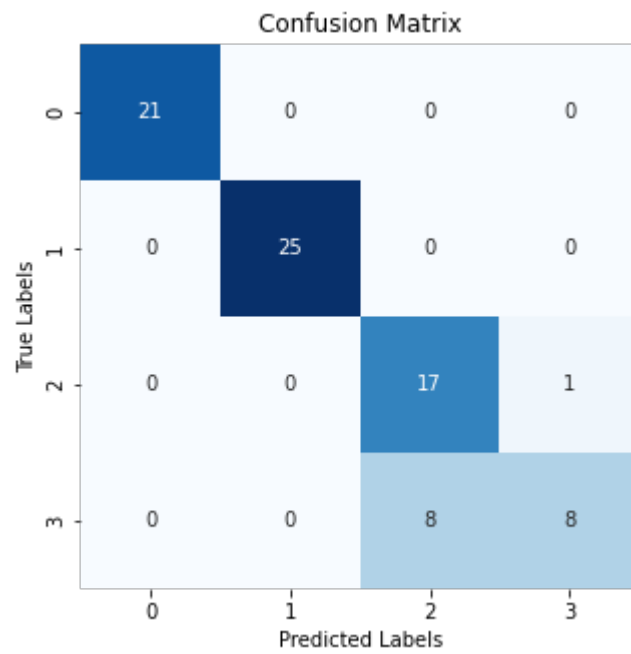
Accuracy: 0.97
Precision: 0.97
Recall: 0.97
F1 Score: 0.97



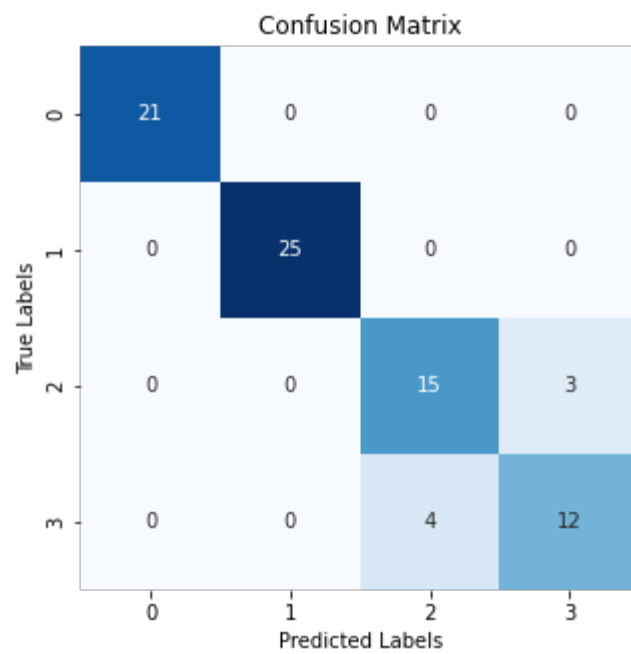
Accuracy: 0.97
Precision: 0.97
Recall: 0.97
F1 Score: 0.97



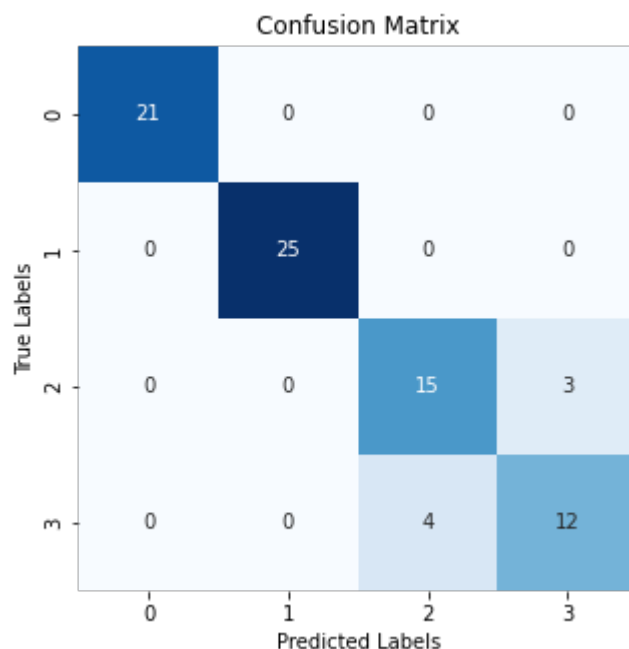
Accuracy: 0.89
Precision: 0.89
Recall: 0.86
F1 Score: 0.86



Accuracy: 0.91
Precision: 0.90
Recall: 0.90
F1 Score: 0.90



Accuracy: 0.91
Precision: 0.90
Recall: 0.90
F1 Score: 0.90



Questions:

1. What is the best performing model trained on the flattened images? Why?
2. What is the best performing model using the HOG features? With which HOG set? Why?
3. What is the best performing model using PCA features? With how many components? Why?
4. Regarding the models trained with PCA features, analyze the SVC performance.
5. For the SVC with PCA features, try the RBF and poly kernels in a new cell, using the three variations of the PCA sets. Compare the performances of these kernels? (Iteration limits might be required for poly kernels)
6. Based on the confusion matrices, can you spot a recurring pattern in the classification results? Relate this pattern with the component plot in task 3 and the original dataset.
7. Explain why higher number of layers/neurons in the MLP does not translate in better accuracy?

```
In [36]: print("Answer 1: The linear SVC model preformed the best because it has the
highest scores of all the models and the confusion matrix preformed the bes
t")

print("\nAnswer 2: There are multiple best performing models using the HOG
features, but I would say the HOG_16_4 set, because it also has the highest
score and the best performing confusion matrix. ")

print("\nAnswer 3: There are again multiple best performing models with equ
al scores of the metric and exactly the same confusion matrices")

print("\nAnswer 4: The SVC performance with PCA features changes with the n
umber of components.")

print("\nAnswer 5: Assuming I did the models correctly I saw that RBF is be
tter than poly. (I'm not sure if I should leave them in or remove them from
the file so I removed them)")

print("\nAnswer 6: Classes 2 and 3 get the lowest accuracy, false positives,
than classes 0 and 1. This is also visable in the PCA because of the big ov
erlap of those 2 classes in the scatterplot. ")

print("\nAnswer 7: Too many layers/neurons result in too many parameters an
d that may result in the model fitting noise instead of generalize the dat
a.")
```

Answer 1: The linear SVC model preformed the best because it has the highest scores of all the models and the confusion matrix preformed the best

Answer 2: There are multiple best performing models using the HOG features, but I would say the HOG_16_4 set, because it also has the highest score and the best performing confusion matrix.

Answer 3: There are again multiple best performing models with equal scores of the metric and exactly the same confusion matrices

Answer 4: The SVC performance with PCA features changes with the number of components.

Answer 5: Assuming I did the models correctly I saw that RBF is better than poly. (I'm not sure if I should leave them in or remove them from the file so I removed them)

Answer 6: Classes 2 and 3 get the lowest accuracy, false positives, than classes 0 and 1. This is also visable in the PCA because of the big overlap of those 2 classes in the scatterplot.

Answer 7: Too many layers/neurons result in too many parameters and that may result in the model fitting noise instead of generalize the data.

Bonus - Pneumonia Classification & Grid Search (2 points)

For this bonus task you will have to discard the samples belonging to classes 0 and 1, and assign the 0 and 1 values to classes 2 and 3. Then you will train a KNN classifier, a SVC and an MLP classifier using the grid search algorithm to identify the best hyperparameters for the new task. Use the following parameter grids:

- knn_param_grid = {'n_neighbors': [1, 3, 5, 7, 9, 11, 13, 15]}
- svm_param_grid = {'kernel': ['linear', 'rbf', 'poly'], 'C': [0.1, 1, 10], 'max_iter':[300]}
- mlp_param_grid = {'hidden_layer_sizes': [(100,), (100, 100), (100, 100, 100)], 'alpha': [0.0001, 0.001, 0.01], 'max_iter':[300]}

Use the identified parameters for the following tasks.

- Train with normal flattened images
- Train with HOG
- Train with PCA

Compare and analyze the best-performing hyperparameter set for each model and training data. Identify the most suitable model and corresponding feature set for this CodeLab. Can you propose another feature engineering method to improve the model performance? Explain.

In []: